

Reviewer Pack — Minimal Rank of Noncommutative L^p Geometric Measures vs DIS...

Entry d605cac0d3b0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 13:52:31 UTC

Minimal Rank of Noncommutative L^p Geometric Measures vs DISJOINTNESS Communication Complexity

Entry ID: d605cac0d3b0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 13:52:31 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity

Statement:

{‘text’: ‘For any partial function $f: \{0, \dots, n-1\} \times \{0, \dots, n-1\} \rightarrow \{-1, +1\}$, define the noncommutative L^p geometric measure $\mu_p(f)$ as the p -th root of the integral over the product space of the $(|f(x,y)|/2)^p$ over all x, y in $\{0, \dots, n-1\}$. Then, the randomized communication complexity of the partial function f is lower-bounded by $O(\mu_p(f)^{1/p})$ for every $1 < p \leq \infty$.’, ‘quantitative’: ‘ $E[X(\text{instance})] = \Theta(\mu_p(\text{DISJ}_n)^{1/p})$ ’}

Rationale (proposer’s reasoning):

{‘text’: ‘Noncommutative L^p geometry has been used to study geometric properties of matrices and functions, which could provide insights into the underlying structure of communication complexity problems. The proposed measure $\mu_p(f)$ generalizes classical geometric measures and may capture the essential characteristics of partial functions that influence their communication complexity.’, ‘quantitative’: “The invariant μ_p captures the essence of a function’s behavior in a noncommutative setting, which could be exploited to derive lower bounds for communication complexity.”}

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 65c76a63cbf69613

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support or falsify the conjecture, calculate $\mu_p(f)$ for each partial function f and compare it to $O(\mu_p(f)^{1/p})$. If at least 80% of seeds show a correlation with a mean communication complexity within ± 3 standard deviations from $O(\mu_p(f)^{1/p})$, the conjecture is supported. Falsification occurs if any seed shows a deviation greater than 10 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_partial_function(n):
        return {(x, y): random.choice([-1, 1]) for x in range(n) for y in range(n)}

    def noncommutative_Lp_measure(f, p):
        n = len(next(iter(f.values())))
        total = 0
        for (x, y), val in f.items():
            total += abs(val / 2) ** p
        return total ** (1 / p)

    def communication_complexity(f):
        # Simplified model: assume  $O(n^2)$  complexity for any function
        return n ** 2

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_partial_function(n)
        mu_p = noncommutative_Lp_measure(f, 2) # Using p=2 for simplicity
        comm = communication_complexity(f)
        results.append((mu_p, comm))

    if not results:
        return {
            "metric_name": "communication_complexity",
```

```

        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

mean_mu_p = sum(mu for mu, _ in results) / len(results)
mean_comm = sum(comm for _, comm in results) / len(results)
std_mu_p = math.sqrt(sum((mu - mean_mu_p) ** 2 for mu, _ in results) / len(results))
std_comm = math.sqrt(sum((comm - mean_comm) ** 2 for _, comm in results) / len(results))

if std_mu_p == 0 or std_comm == 0:
    return {
        "metric_name": "communication_complexity",
        "metric_value": None,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "std_deviation_zero"
    }

correlation = sum((mu_p - mean_mu_p) * (comm - mean_comm) for mu_p, comm in results) / (len(re

return {
    "metric_name": "communication_complexity",
    "metric_value": correlation,
    "instances_tested": len(results),
    "conjecture_holds": abs(correlation) > 0.3, # Threshold for support
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["metric_value"] is not None for result in results):
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
        elif any(abs(result["metric_value"]) > 10 * std_value for result in results):
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if abs(result["metric_value"]) > 10 * std_value)
            print(f"RESULT: FALSIFIED counterexample=\"std_deviation\" first_failing_seed={first_failing_seed}")
        else:
            print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
    else:
        print("RESULT: INCONCLUSIVE empty_results")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_047f7414.py", line 83, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_047f7414.py", line 40, in run_trial
    mu_p = noncommutative_Lp_measure(f, 2) # Using p=2 for simplicity
           ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_047f7414.py", line 25, in noncommutative_Lp_measure
    n = len(next(iter(f.values())))
           ~~~~~~
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13597
2	preregistration	ollama_remote	glm4:latest	0	0	5859
3	novelty	ollama_remote	glm4:latest	0	0	5012
4	novelty	ollama_remote	glm4:latest	0	0	5177
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15081
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9676
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8933
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11043
9	judge	ollama_remote	glm4:latest	0	0	8399

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 82777 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d605cac0d3b0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d605cac0d3b0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d605cac0d3b0.tar.gz` (if generated)